

Universidad Rafael Landívar

Facultad de Ingeniería

Licenciatura en Ingeniería en Informática y Sistemas

**Manual Técnico y Manual de Usuario - Aplicación de Escritorio con Gestión de
Configuración de Usuario**

Curso: Manejo e Implementación de Archivos

Sección: 01

Estudiante: María Belén Mendoza Garcia

Carnet: 1517225

Guatemala, septiembre 2026

ÍNDICE

MANUAL TÉCNICO

1. Descripción del proyecto
2. Formato de configuración
 - 2.1. Formato elegido: JSON
 - 2.2. Alternativa descartada: YAML
 - 2.3. Comparación técnica
3. Estructura del proyecto
4. Explicación de la implementación
 - 4.1. Lectura de la configuración
 - 4.2. Escritura y respaldo
 - 4.3. Codificación de caracteres
5. Manejo de errores
 - 5.1. Archivo ausente
 - 5.2. Archivo corrupto
 - 5.3. Falta de permisos

MANUAL DE USUARIO

1. Inicio del programa
2. Ventana principal
3. Ventana de configuración
 - 3.1. Configuración general
 - 3.2. Personalización de textos
4. Archivo de configuración generado

5. Evidencias de funcionamiento

6. Finalización

Conclusiones

Referencias

MANUAL TÉCNICO

1. Descripción del proyecto

El proyecto consiste en una aplicación de escritorio desarrollada en Python mediante Tkinter. Su objetivo es permitir que el usuario modifique diferentes preferencias de la interfaz y que estas se mantengan después de cerrar y volver a abrir el programa.

La aplicación permite cambiar el nombre de usuario, el tema, el idioma, el color de la barra de menú, la fotografía de perfil y algunos estilos de texto. Para conservar estos cambios se utiliza un archivo de configuración en formato JSON.

Además del archivo principal, se utiliza un archivo temporal y un archivo de respaldo para realizar el guardado de una forma más segura. También se manejan los casos en los que el archivo no existe, está corrupto o no se puede leer o escribir por falta de permisos.

2. Formato de configuración

2.1. Formato elegido: JSON

Se eligió JSON para almacenar la configuración porque los datos utilizados por la aplicación son principalmente pares de clave y valor. Esto permite que el archivo sea fácil de leer, modificar y procesar desde Python.

Otra ventaja es que JSON cuenta con soporte directo en Python mediante la biblioteca estándar json, por lo que no es necesario instalar una herramienta adicional para leer o guardar la configuración.

Para este proyecto también es importante la codificación de caracteres. El archivo se maneja en UTF-8 para permitir valores como nombres con tildes y la letra ñ sin tener que cambiar la estructura del archivo.

```
{  
  "nombre_usuario": "José Muñoz",  
  "tema_interfaz": "claro",  
  "idioma": "es-ES",  
  "tamano_fuente": 11,  
  "color_barra_menu": "#171c26",  
  "color_letra": "#ffffff",  
  "foto_perfil": ""  
}
```

2.2. Alternativa descartada: YAML

Como alternativa se consideró YAML. Este formato también es legible para una persona y puede representar configuraciones de forma sencilla. Sin embargo, para este proyecto se descartó porque JSON ya tiene soporte directo en Python y permite mantener una estructura clara sin agregar una dependencia externa.

La elección no se basa únicamente en que JSON sea más corto o más fácil de leer. Para la aplicación también se tomó en cuenta la facilidad de parseo, la cantidad de dependencias y la posibilidad de validar rápidamente si el contenido tiene una estructura correcta.

2.3. Comparación técnica

Característica o criterio	JSON	YAML
Tamaño	Adecuado para una configuración pequeña; la estructura utiliza poca información adicional.	Puede ser similar o menor en archivos sencillos, pero depende de la forma de escritura.
Legibilidad	Alta; utiliza claves, valores, listas y objetos con una estructura definida.	Alta; su sintaxis suele ser cómoda para configuraciones escritas manualmente.
Facilidad de parseo	Alta; Python incluye la biblioteca estándar json.	Requiere una biblioteca adicional para realizar el parseo en Python.
Robustez ante corrupción	Permite detectar errores de sintaxis al intentar cargar el archivo y manejar la excepción correspondiente.	También puede detectar errores de sintaxis, pero depende del parser utilizado.
Codificación	Permite trabajar con UTF-8 y conservar tildes y ñ.	También permite Unicode, pero no aporta una ventaja necesaria para este proyecto.
Dependencias	No requiere una dependencia externa para leer y escribir el archivo.	Normalmente requiere una biblioteca externa en Python.

Por las características de la aplicación, JSON resulta suficiente para almacenar la configuración y permite mantener el código de lectura y escritura sencillo.

3. Estructura del proyecto

El proyecto se encuentra organizado de la siguiente manera:

```
Proyecto-1---Aplicacion-de-Escritorio-con-Gestion-de-Configuracion-de-Usuario
|
├── README.md
|
├── data
|   ├── config.json
|   ├── config.bak
|   └── config.tmp
|
├── docs
|
└── src
    ├── __init__.py
    ├── config_manager.py
    └── main.py
```

README.md

Contiene información general del proyecto, instrucciones básicas y datos necesarios para comprender su propósito y ejecución.

data/config.json

Es el archivo principal de configuración. Guarda los valores seleccionados por el usuario, como el nombre, el tema, el idioma, los colores y los estilos de los textos.

data/config.bak

Es el archivo de respaldo. Conserva una copia de la configuración anterior antes de realizar un nuevo guardado.

data/config.tmp

Es el archivo temporal utilizado durante el proceso de guardado seguro. Contiene momentáneamente la nueva información antes de sustituir el archivo principal.

src/main.py

Contiene la interfaz gráfica, la ventana principal, la ventana de configuración, los botones y las funciones relacionadas con la interacción del usuario.

src/config_manager.py

Contiene las funciones encargadas de leer y guardar la configuración en formato JSON y de manejar los errores relacionados con los archivos.

src/__init__.py

Permite identificar la carpeta src como un paquete de Python.

4. Explicación de la implementación

4.1. Lectura de la configuración

Al iniciar la aplicación, se intenta leer el archivo data/config.json. Si el archivo existe y contiene un JSON válido, sus valores se utilizan para configurar la interfaz. Si el archivo no existe, se cargan los valores predeterminados.

La lectura también debe controlar los errores de formato. Cuando el contenido no corresponde a un JSON válido, la aplicación informa el problema y utiliza nuevamente los valores predeterminados para poder continuar.

```
with open(CONFIG_FILE, "r", encoding="utf-8") as archivo:
    configuracion = json.load(archivo)
```

El uso de `encoding="utf-8"` permite que la lectura conserve correctamente caracteres como á, é, í, ó, ú y ñ.

4.2. Escritura y respaldo

Para guardar la configuración se utiliza un archivo temporal. Primero se escribe la nueva información en config.tmp. Si ya existe config.json, este se conserva como config.bak y después el archivo temporal pasa a ocupar el lugar del archivo principal.

De esta manera, el archivo principal no se modifica directamente desde el comienzo de la operación. Si ocurre un problema durante el proceso, se puede conservar la configuración anterior en el archivo de respaldo.

```
with open(TEMP_FILE, "w", encoding="utf-8") as archivo:
    json.dump(configuracion, archivo, ensure_ascii=False, indent=4)
```

```
if os.path.exists(CONFIG_FILE):  
    os.replace(CONFIG_FILE, BACKUP_FILE)  
  
os.replace(TEMP_FILE, CONFIG_FILE)
```

El parámetro `ensure_ascii=False` es importante para que las tildes y la ñ puedan conservarse directamente en el archivo en lugar de convertirse en secuencias Unicode.

El proceso también contempla la eliminación del archivo temporal cuando ocurre un error, evitando dejar archivos temporales innecesarios.

4.3. Codificación de caracteres

La configuración se guarda utilizando UTF-8. Para comprobar que la codificación funciona correctamente, se puede utilizar un valor como "José Muñoz" dentro del archivo. Al abrir el archivo debe observarse el texto con sus caracteres originales.

```
{  
    "nombre_usuario": "José Muñoz",  
    "idioma": "es-ES"  
}
```

Esta prueba permite comprobar que la aplicación no pierde ni cambia las tildes o la letra ñ al guardar y volver a cargar la configuración.

5. Manejo de errores

La aplicación contempla los tres casos solicitados para las pruebas de configuración: archivo ausente, archivo corrupto y falta de permisos. En cada caso se debe comprobar el comportamiento real de la aplicación y conservar una captura o registro como evidencia.

5.1. Archivo ausente

Si data/config.json no existe al iniciar la aplicación, el programa utiliza los valores predeterminados. Esto permite iniciar la aplicación aunque todavía no se haya creado un archivo de configuración.

```
if not os.path.exists(CONFIG_FILE):  
    return DEFAULT_CONFIG.copy()
```

Evidencia requerida: colocar aquí la captura o log de la prueba realizada con config.json ausente. La evidencia debe mostrar que la aplicación inició utilizando los valores predeterminados.

5.2. Archivo corrupto

Si config.json existe pero contiene información que no corresponde a un JSON válido, la lectura genera un error. La aplicación debe manejarlo y continuar con los valores predeterminados en lugar de cerrarse.

```
try:  
    with open(CONFIG_FILE, "r", encoding="utf-8") as archivo:  
        configuracion = json.load(archivo)  
except json.JSONDecodeError:  
    print("Error: el archivo de configuración está corrupto.")  
    configuracion = DEFAULT_CONFIG.copy()
```

Evidencia requerida: colocar aquí la captura o log de la prueba realizada con un config.json corrupto. Debe observarse el mensaje de error y que la aplicación pueda continuar.

5.3. Falta de permisos

Cuando el programa no tiene permisos suficientes para leer o escribir el archivo de configuración, se produce un error de acceso. Este caso debe manejarse mostrando un mensaje y evitando que la aplicación termine de forma inesperada.

```
try:
    # operación de lectura o escritura
    ...
except PermissionError:
    print("Error: no se tienen permisos para acceder al archivo.")
```

Evidencia requerida: colocar aquí la captura o log de la prueba realizada con permisos restringidos. Debe mostrarse el mensaje correspondiente al problema de acceso.

Los tres casos deben probarse sobre el proyecto real. Las capturas o logs que se coloquen en este apartado deben corresponder a las pruebas realizadas durante el desarrollo.

MANUAL DE USUARIO

1. Inicio del programa

Para iniciar la aplicación se debe abrir una terminal ubicada en la carpeta principal del proyecto y ejecutar el siguiente comando:

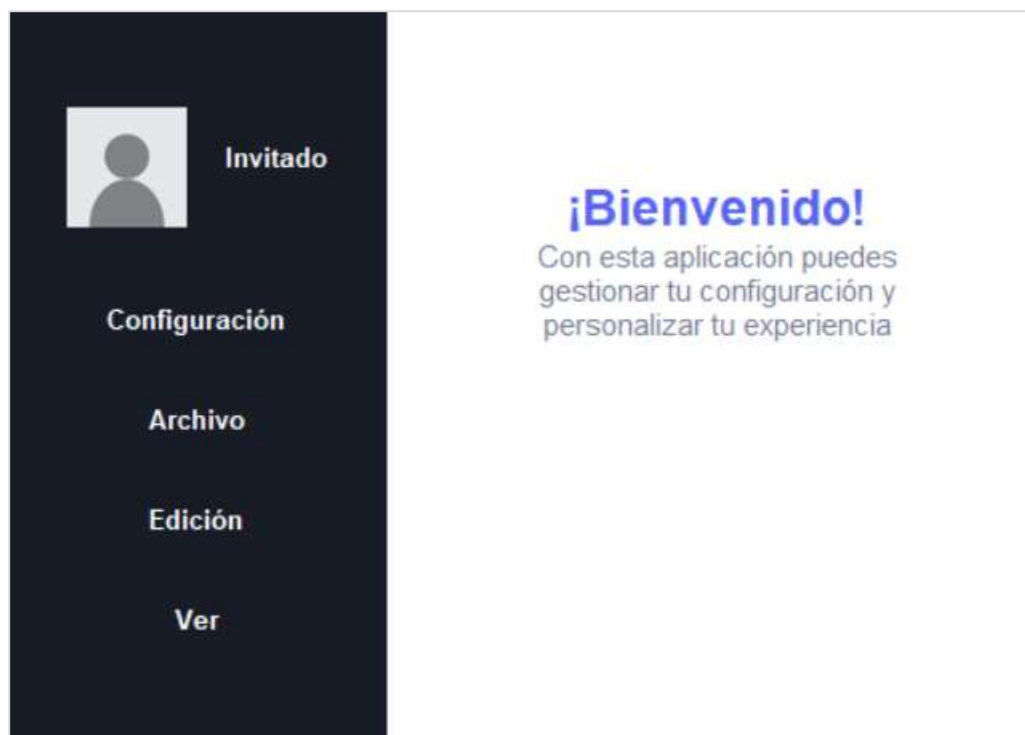
```
py -m src.main
```

Al iniciar, la aplicación intenta cargar la configuración guardada. Si no existe un archivo de configuración válido, se utilizan los valores predeterminados.

2. Ventana principal

Después de iniciar se muestra la ventana principal de la aplicación. En ella se presenta el usuario y los elementos visuales configurados, además de la barra de menú.

La opción Configuración o Settings permite abrir la ventana donde se pueden modificar las preferencias. Las opciones Archivo, Edición y Ver se mantienen como opciones simuladas cuando así lo establece el proyecto.



3. Ventana de configuración

La ventana de configuración permite modificar las preferencias sin editar directamente el archivo JSON. Está organizada en una sección de configuración general y otra de personalización de textos.

Configuración

Personalización de textos

Elemento: Texto de bienvenida

Tamaño: 19

Color de texto: [Blue]

☒ Negrita

☐ Cursiva

Aplicar estilo

Configuración general

Nombre de usuario: Invitado

Tema: claro

Idioma: es-ES

Color de barra de menú: [Black]

Foto de perfil:

Foto seleccionada: C:\Users\Dell\Desktop\URL\4to Semestre\Manejo e Imples\Proyecto 1\guest.png

Seleccionar foto

Guardar configuración

3.1. Configuración general

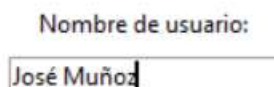
En esta sección se pueden modificar las siguientes opciones:

- Nombre de usuario.
- Tema claro u oscuro.
- Idioma español o inglés.
- Color de la barra de menú.
- Fotografía de perfil.

Para guardar un cambio, se debe seleccionar la opción correspondiente, realizar la modificación y presionar Guardar. Al finalizar, la aplicación aplica la configuración seleccionada y la conserva en el archivo JSON.

3.1.1. Modificar el nombre de usuario

Para cambiar el nombre mostrado en la aplicación se debe seleccionar el campo correspondiente, escribir el nuevo nombre y presionar Guardar.

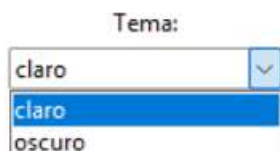


Nombre de usuario:

Nota. La figura muestra el campo utilizado para cambiar el nombre del usuario.

3.1.2. Cambiar el tema

Para cambiar el tema de la aplicación se debe seleccionar la opción Tema y elegir entre claro u oscuro. Después se debe presionar Guardar para conservar el cambio.



Tema:

claro	▼
claro	
oscuro	

Nota. La figura muestra las opciones disponibles para seleccionar el tema de la aplicación.

3.1.3. Cambiar el idioma

Para cambiar el idioma se debe seleccionar la opción Idioma y elegir entre español o inglés. Después se debe presionar Guardar.



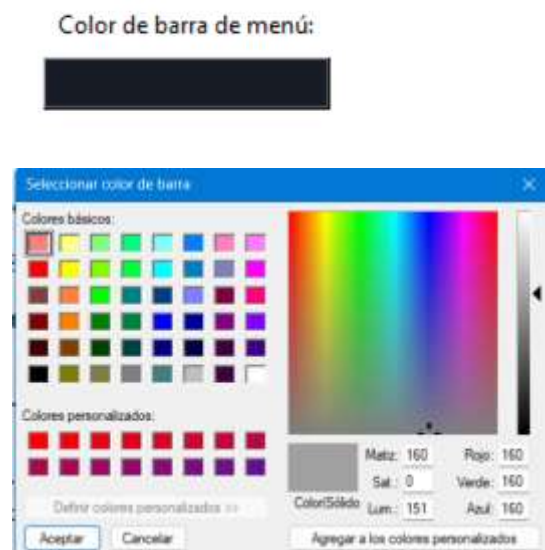
Idioma:

es-ES	▼
es-ES	
en-US	

Nota. La figura muestra las opciones disponibles para seleccionar el idioma.

3.1.4. Cambiar el color de la barra de menú

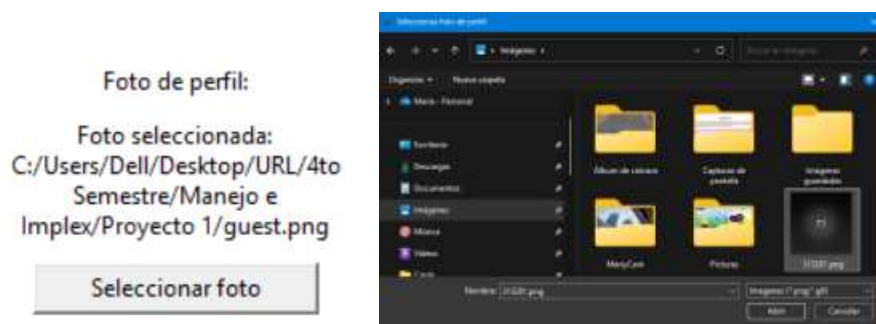
Para modificar el color de la barra de menú se debe seleccionar la opción correspondiente y elegir el color deseado. Después se debe presionar Guardar para conservar el cambio.



Nota. La figura muestra la herramienta utilizada para seleccionar el color de la barra de menú.

3.1.5. Seleccionar una fotografía de perfil

Para agregar o cambiar la fotografía de perfil se debe presionar la opción para seleccionar una fotografía y buscar una imagen compatible en el equipo. Después se debe presionar Guardar.



Nota. La figura muestra el proceso utilizado para seleccionar la imagen de perfil.

3.2. Personalización de textos

Esta sección permite seleccionar un elemento de la interfaz y modificar su apariencia. Las opciones disponibles incluyen tamaño de fuente, color, negrita y cursiva.

Los elementos que pueden personalizarse incluyen:

- Texto de bienvenida.
- Descripción.
- Nombre de usuario.
- Configuración.
- Archivo.
- Edición.
- Ver.

Después de seleccionar las opciones deseadas, se puede utilizar Aplicar para observar el cambio y Guardar para conservarlo.

The image displays two side-by-side screenshots of a 'Personalización de textos' (Text Customization) dialog box. Both screenshots show the 'Elemento:' dropdown menu set to 'Texto de bienvenida'. The left screenshot shows the dropdown menu open, listing options: 'Texto de bienvenida', 'Descripción', 'Nombre de usuario', 'Botón Configuración', 'Botón Archivo', 'Botón Edición', and 'Botón Ver'. Below the menu are checkboxes for 'Negrita' (checked) and 'Cursiva' (unchecked), and an 'Aplicar estilo' button. The right screenshot shows the 'Elemento:' dropdown set to 'Texto de bienvenida'. Below it are a 'Tamaño:' spinner set to 19, a 'Color de texto:' color picker showing blue, and checkboxes for 'Negrita' (checked) and 'Cursiva' (unchecked), with an 'Aplicar estilo' button at the bottom.

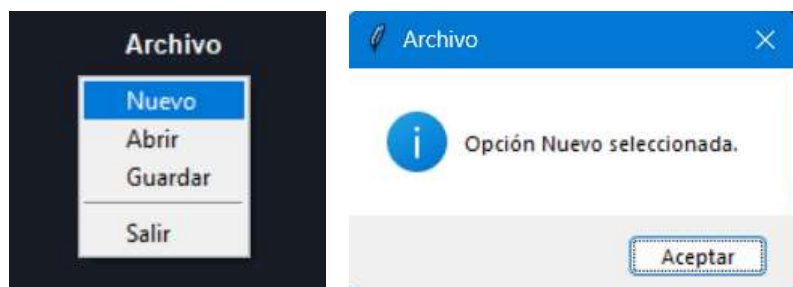
Nota. La figura muestra las opciones para cambiar el tamaño, color y estilo de los textos.

4. Menús de la aplicación

La ventana principal también cuenta con las opciones Archivo, Edición y Ver. Estas opciones se mantienen como opciones simuladas cuando así lo establece el proyecto.

4.1. Menú Archivo

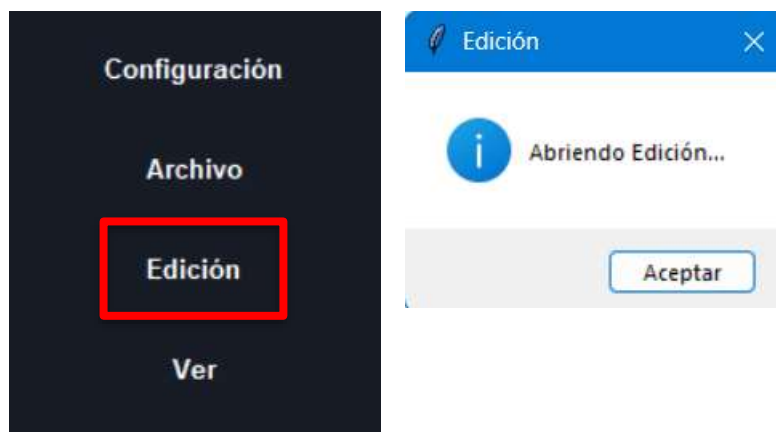
Al seleccionar Archivo se muestran las opciones disponibles en el menú. Estas opciones son simuladas y muestran un mensaje informativo cuando se seleccionan.



Nota. La figura muestra el menú Archivo desplegado con sus opciones disponibles.

4.2. Menú Edición

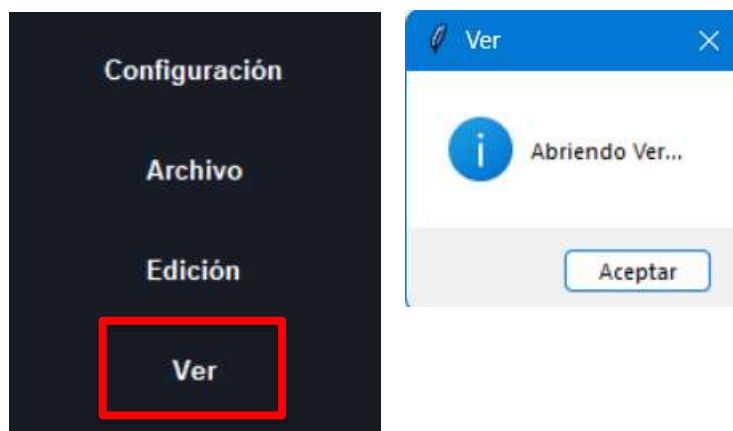
La opción Edición es simulada. Al seleccionarla, la aplicación muestra un mensaje indicando que se seleccionó dicha opción.



Nota. La figura muestra el menú Edición y el funcionamiento de su opción simulada.

4.3. Menú Ver

La opción Ver también es simulada. Al seleccionarla, la aplicación muestra un mensaje informativo.



Nota. La figura muestra el menú Ver y el mensaje mostrado por la aplicación.

5. Archivo de configuración generado

El archivo generado por la aplicación se encuentra en `data/config.json`. Su contenido depende de las opciones seleccionadas por el usuario.

Un ejemplo que permite comprobar el manejo de caracteres especiales es:

```
{
  "nombre_usuario": "José Muñoz",
  "tema_interfaz": "claro",
  "idioma": "es-ES",
  "tamano_fuente": 11,
  "color_barra_menu": "#171c26",
  "color_letra": "#ffffff",
  "foto_perfil": ""
}
```

El archivo utiliza una estructura de clave y valor. Los valores de texto se almacenan como cadenas, mientras que el tamaño de fuente se almacena como un número.

Archivo config.json

```

1  {
2    "nombre_usuario": "José Muñoz",
3    "tema_interfaz": "claro",
4    "idioma": "es-ES",
5    "tamano_fuente": 11,
6    "color_barra_menu": "#171c26",
7    "color_letra": "ffffff",
8    "foto_perfil": "C:/Users/Dell/Pictures/313281.png",
9    "estilos_textos": {
10     "bienvenida": {
11       "fuente": "Arial",
12       "tamano": 19,
13       "color": "#5085f2",
14       "negrita": true,
15       "cursiva": false
16     },
17     "descripcion": {
18       "fuente": "Arial",
19       "tamano": 11,
20       "color": "#777f95",
21       "negrita": false,
22       "cursiva": false
23     },
24     "nombre_usuario": {
25       "fuente": "Helvetica",
26       "tamano": 18,
27       "color": "ffffff",
28       "negrita": true,
29       "cursiva": false
30     },
31     "configuracion": {
32       "fuente": "Arial",
33       "tamano": 10,
34       "color": "ffffff",
35       "negrita": true,
36       "cursiva": false

```

```

37     },
38     "archivo": {
39       "fuente": "Arial",
40       "tamano": 10,
41       "color": "ffffff",
42       "negrita": true,
43       "cursiva": false
44     },
45     "edicion": {
46       "fuente": "Arial",
47       "tamano": 18,
48       "color": "ffffff",
49       "negrita": true,
50       "cursiva": false
51     },
52     "vie": {
53       "fuente": "Arial",
54       "tamano": 10,
55       "color": "ffffff",
56       "negrita": true,
57       "cursiva": false
58     },
59     "color_fondo": "#4f607b",
60     "color_panel": "ffffff",
61     "color_tarjeta": "ffffff"
62   }
63 }

```

Nota. La figura muestra el archivo de configuración generado por la aplicación, incluyendo caracteres con tildes y ñ.

El archivo **config.bak** contiene la configuración anterior cuando se realiza un nuevo guardado y **config.tmp** se utiliza únicamente durante la operación de escritura segura.

Archivo config.bak

```
data > E config.bak
1 {
2   "nombre_usuario": "Esau",
3   "tema_interfaz": "claro",
4   "idioma": "es-ES",
5   "tamano_fuente": 11,
6   "color_barra_menu": "#171c26",
7   "color_letra": "#ffffff",
8   "foto_perfil": "C:/Users/Dell/Pictures/313281.png",
9   "estilos_textos": {
10    "bienvenida": {
11      "fuente": "Arial",
12      "tamano": 19,
13      "color": "#5865f2",
14      "negrita": true,
15      "cursiva": false
16    },
17    "descripcion": {
18      "fuente": "Arial",
19      "tamano": 11,
20      "color": "#777f95",
21      "negrita": false,
22      "cursiva": false
23    },
24    "nombre_usuario": {
25      "fuente": "Helvetica",
26      "tamano": 10,
27      "color": "#ffffff",
28      "negrita": true,
29      "cursiva": false
30    },
31    "configuracion": {
32      "fuente": "Arial",
33      "tamano": 10,
34      "color": "#ffffff",
35      "negrita": true,
36      "cursiva": false
37    },
38    "archivo": {
39      "fuente": "Arial",
40      "tamano": 10,
41      "color": "#ffffff",
42      "negrita": true,
43      "cursiva": false
44    },
45    "edicion": {
46      "fuente": "Arial",
47      "tamano": 10,
48      "color": "#ffffff",
49      "negrita": true,
50      "cursiva": false
51    },
52    "ver": {
53      "fuente": "Arial",
54      "tamano": 10,
55      "color": "#ffffff",
56      "negrita": true,
57      "cursiva": false
58    }
59  },
60   "color_fondo": "#4a69bd",
61   "color_panel": "#ffffff",
62   "color_tarjeta": "#ffffff"
63 }
```

```
37 },
38 "archivo": {
39   "fuente": "Arial",
40   "tamano": 10,
41   "color": "#ffffff",
42   "negrita": true,
43   "cursiva": false
44 },
45 "edicion": {
46   "fuente": "Arial",
47   "tamano": 10,
48   "color": "#ffffff",
49   "negrita": true,
50   "cursiva": false
51 },
52 "ver": {
53   "fuente": "Arial",
54   "tamano": 10,
55   "color": "#ffffff",
56   "negrita": true,
57   "cursiva": false
58 },
59 },
60 "color_fondo": "#4a69bd",
61 "color_panel": "#ffffff",
62 "color_tarjeta": "#ffffff"
63 }
```

Nota. La figura muestra el respaldo creado durante el proceso de guardado.

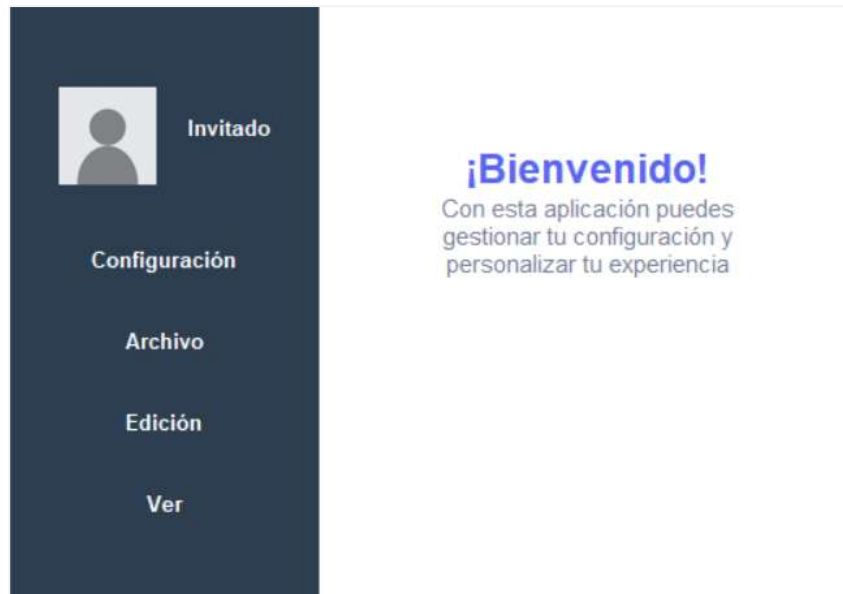
6. Pruebas de manejo de errores

Como parte del funcionamiento de la aplicación se realizaron pruebas para comprobar el comportamiento ante diferentes situaciones relacionadas con el archivo de configuración.

6.1. Archivo ausente

Cuando el archivo `config.json` no está presente, la aplicación utiliza los valores predeterminados para iniciar correctamente.

Prueba de archivo ausente

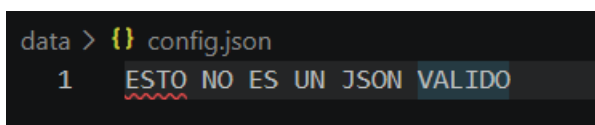


Nota. La figura muestra la evidencia de la prueba realizada cuando el archivo de configuración no está presente.

6.2. Archivo corrupto

Cuando el contenido del archivo no tiene un formato JSON válido, la aplicación informa del problema y utiliza los valores predeterminados para continuar funcionando.

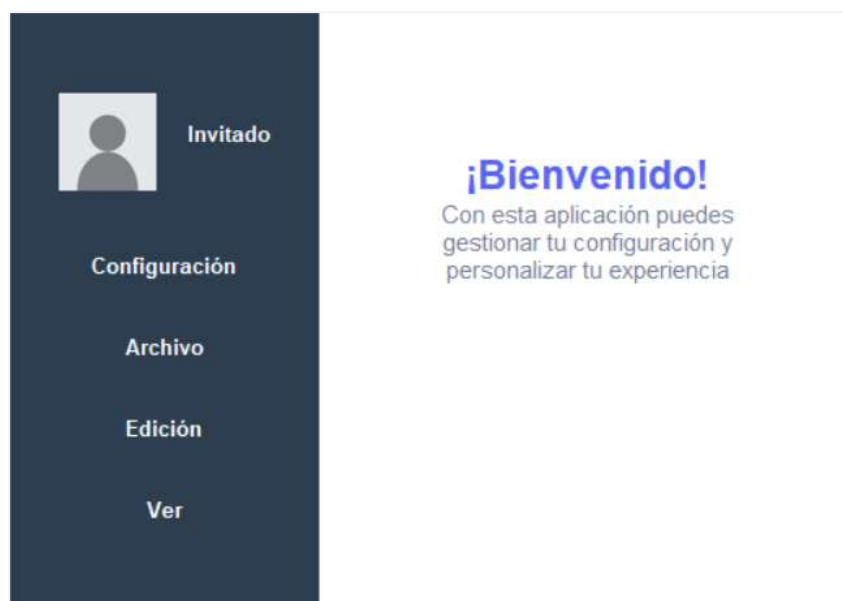
Prueba de archivo corrupto



Nota. La figura muestra un archivo json corrupto



Nota. La figura muestra el mensaje o registro obtenido al utilizar un archivo JSON con contenido inválido.



Nota. La figura muestra la aplicación iniciada con los valores predeterminados

6.3. Falta de permisos

Cuando el programa no tiene los permisos necesarios para leer o escribir el archivo de configuración, se informa que no fue posible realizar la operación.

Prueba de falta de permisos

```
PS C:\Users\Dell\Desktop\URL\4to Semestre\Manejo e Implex\Proyecto 1\Proyecto-1---Aplicaci-n-de-Escritorio-con-Gesti-n-de-Configuraci-n-de-Usuario> icacls data\config.json /deny "${env:USERNAME}:R" <<< Denegar permisos a usuario
archivo procesado: data\config.json
Se procesaron correctamente 1 archivos; error al procesar 0 archivos
PS C:\Users\Dell\Desktop\URL\4to Semestre\Manejo e Implex\Proyecto 1\Proyecto-1---Aplicaci-n-de-Escritorio-con-Gesti-n-de-Configuraci-n-de-Usuarios> .\main
No tienes permiso para leer el archivo de configuracion.
```

Nota. La figura muestra el mensaje o registro obtenido al intentar realizar una operación sin los permisos necesarios. (Los permisos pueden otorgarse nuevamente con el comando: `icacls data\config.json /remove:d "${env:USERNAME}"`)



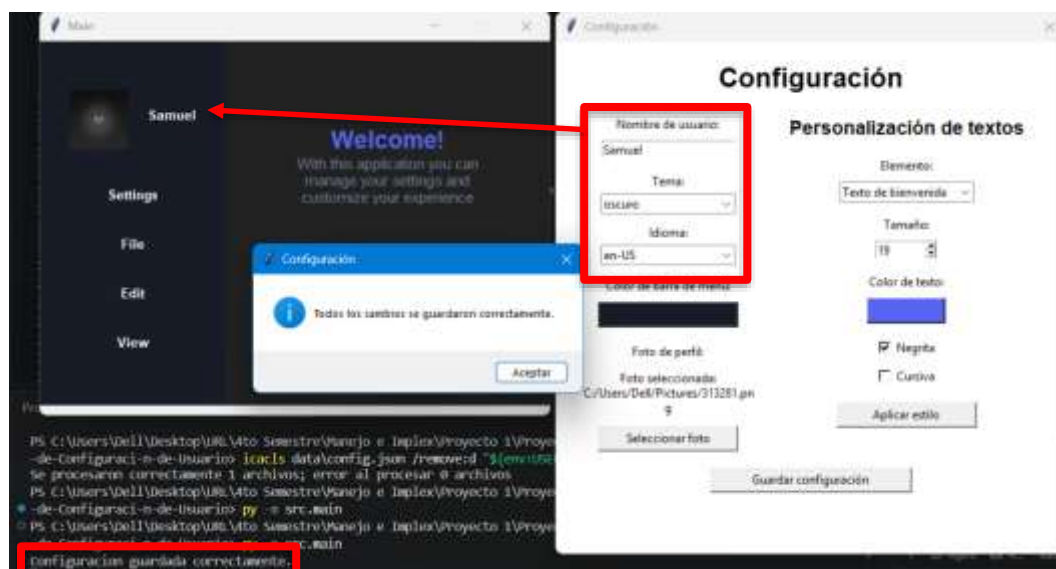
Nota. La figura muestra la aplicación iniciada con los valores predeterminados al no contar con los permisos de usuario.

7. Persistencia de la configuración

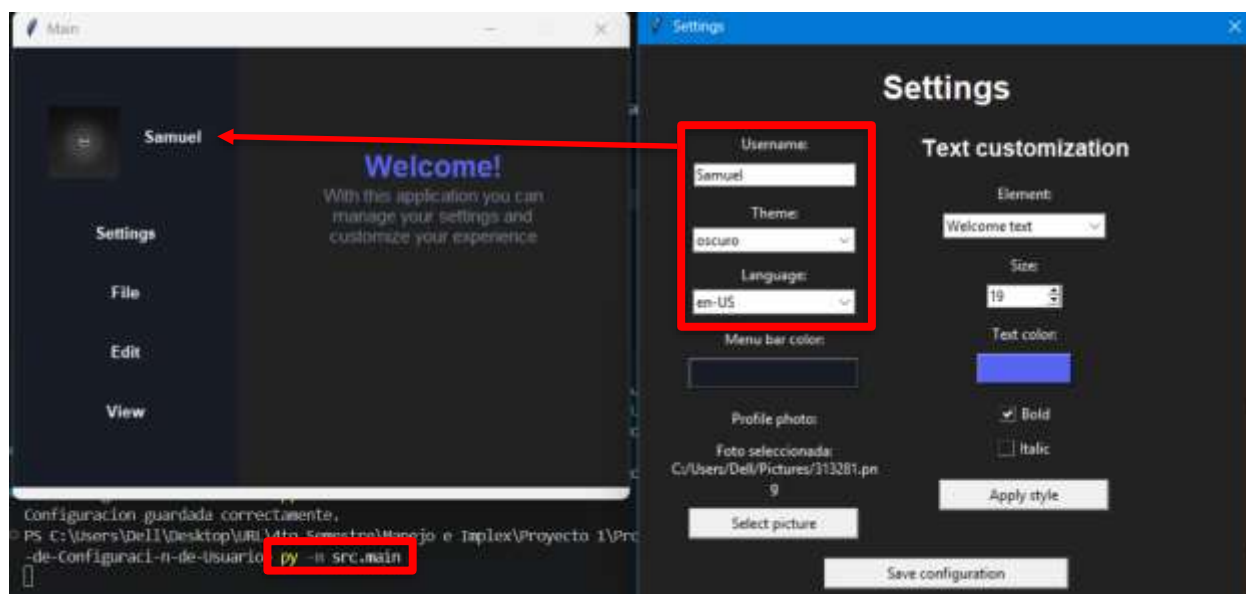
Después de realizar diferentes cambios en la configuración, se puede cerrar la aplicación y volver a ejecutarla para comprobar que los valores seleccionados se mantienen.

Esta prueba permite comprobar que la configuración fue guardada correctamente y que la aplicación puede utilizarla nuevamente al iniciar.

Persistencia de la configuración



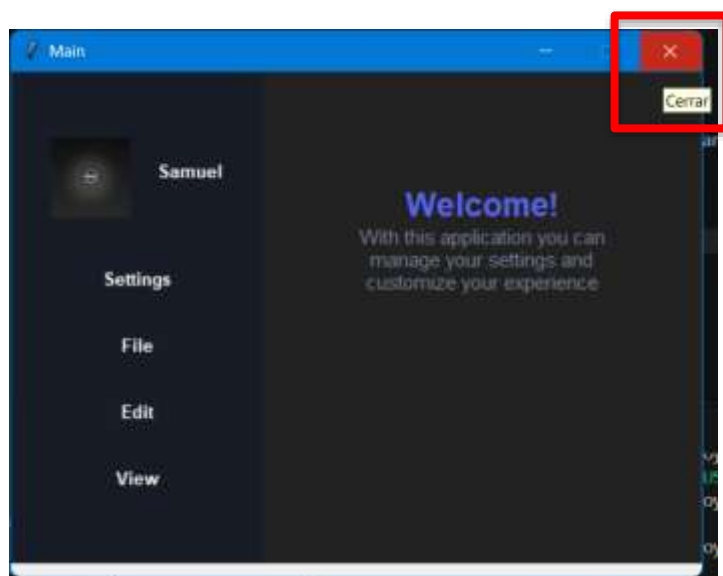
Nota. La figura muestra la nueva configuración guardada correctamente por el usuario.



Nota. La figura muestra la aplicación después de cerrarla y volverla a abrir, comprobando que los cambios realizados anteriormente se conservaron.

8. Finalización

Para finalizar la aplicación se debe cerrar la ventana principal utilizando la opción de salida disponible en la interfaz. Antes de cerrar, se recomienda guardar cualquier cambio realizado en la configuración.



Conclusiones

Desarrollar la aplicación permitió poner en práctica lo aprendido sobre interfaces gráficas, manejo de archivos y guardado de información. También se pudo comprobar que guardar la configuración del usuario fuera del código hace que sea más fácil cambiar sus preferencias sin tener que modificar directamente el programa.

Se eligió JSON porque es un formato sencillo y se adapta bien a los datos que necesita la aplicación. Además, Python permite trabajar fácilmente con este tipo de archivos. También se utilizó UTF-8 para que los textos con caracteres especiales se puedan guardar y mostrar correctamente.

Para guardar la configuración se utilizó un archivo temporal y uno de respaldo, lo que ayuda a que el proceso de guardado sea más seguro y se pueda conservar la configuración anterior en caso de que ocurra algún problema con el sistema. También se probaron diferentes situaciones, como cuando el archivo no existe, cuando está corrupto o cuando no se tienen permisos para acceder a él, y se comprobó que la aplicación puede responder a estos problemas.

Finalmente, con las pruebas y capturas realizadas se pudo comprobar que la aplicación funciona correctamente y que los cambios realizados en la configuración se pueden guardar y mantener al volver a abrir el programa.

Este proyecto ayudó a entender mejor cómo se puede manejar la información de una aplicación de una forma más ordenada y segura. También permitió aprender de los errores que pueden aparecer al trabajar con archivos y ver la importancia de probar diferentes situaciones antes de dar por terminado un programa. Con esto se obtuvo una mejor idea de cómo aplicar estos conocimientos en otros proyectos.

Referencias

Acsany, P. (2025, 20 de agosto). *Working with JSON data in Python*. Real Python.
<https://realpython.com/python-json/>

Python Software Foundation. (s. f.). *json — JSON encoder and decoder*. Python Documentation.
<https://docs.python.org/3/library/json.html>

Python Software Foundation. (s. f.). *os — Miscellaneous operating system interfaces*. Python Documentation.
<https://docs.python.org/3/library/os.html>

Python Software Foundation. (s. f.). *tkinter — Python interface to Tcl/Tk*. Python Documentation.
<https://docs.python.org/3/library/tkinter.html>

Pillow. (s. f.). *Pillow documentation*. <https://pillow.readthedocs.io/>

Tkinter Designer. (s. f.). *Tkinter Designer: Free online Tkinter GUI builder*.
<https://visualtkinter.com/>